

HW #1 Solutions

CSEE W3827 - Fundamentals of Computer Systems Spring 2026

Prof. Rubenstein

Topics: binary number representations (2-complement, 1-complement, floating point)

Note that this homework has 7 problems and is 5 pages long.

It's a bit long and tedious. Not all HW will be like this.

1. Assume an architecture where all numbers are to be represented using 8 bits. What are the (base 10) values of the 8-bit binary numbers when interpreted using (i) unsigned, (ii) signed magnitude, (iii) 1's complement, (iv) 2's complement form:

(a) 00110011

Answer: Since the most significant bit is 0, the value is positive in all forms, and thus identical:
 $= 32 + 16 + 2 + 1 = 51$
i) 51
ii) 51
iii) 51
iv) 51

(b) 10000000

Answer:
i) $2^8 = 128$
ii) $-X = 0$ so $X = 0$
iii) Flip all bits, then $-X = 01111111 = 127$, so $X = -127$.
iv) $X = -2^{(8-1)} = -2^7 = -128$.

(c) 11111111

Answer:
i) $= 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255$
ii) $-X = 01111111 = 64 + 32 + 16 + 8 + 4 + 2 + 1 = 127$, so $X = -127$.
iii) Flip all bits, then $-X = 00000000 = 0$, so $X = 0$.
iv) Flip all bits and add 1. $-X = 00000000 + 1 = 1$, so $X = -1$.

(d) 10011011

Answer:
i) $= 128 + 16 + 8 + 2 + 1 = 155$
ii) $-X = 0011011 = 16 + 8 + 2 + 1 = 27$, so $X = -27$.
iii) $-X = 1100100 = 64 + 32 + 4 = 100$, so $X = -100$.
iv) If the number is negative, 2's complement = 1's complement - 1, so $X = -101$

(e) 10001010

Answer:
i) $128 + 8 + 2 = 138$
ii) $-(8 + 2) = -10$
iii) $-1110101 = -(64 + 32 + 16 + 4 + 1) = -117$
iv) $-128 + 8 + 2 = -118$

2. Convert the following (base 10) numbers to their binary representation using 8-bit (i) signed magnitude, (ii) 1's complement, (iii) 2's complement forms:

(a) -1

Answer:

i) $-X = 00000001$, so $X = 10000001$.

ii) $-X = 00000001$, flip all bits, so $X = 11111110$.

iii) $-X = 00000001$, flip all bits plus 1, so $X = 11111111$.

(b) -15

Answer:

i) $-X = 15 = 8 + 4 + 2 + 1 = 00001111$, so $X = 10001111$.

ii) $-X = 00001111$, flip all bits, so $X = 11110000$.

iii) 1's complement plus 1, so $X = 11110001$.

(c) -67

Answer:

i) $-X = 67 = 64 + 2 + 1 = 01000011$, so $X = 11000011$.

ii) $-X = 01000011$, flip all bits, so $X = 10111100$.

iii) 1's complement plus 1, so $X = 10111101$.

(d) -127

Answer:

i) 11111111

ii) 10000000

iii) 10000001

3. For each pair of x and y below, convert x and y to their 8-bit 2's-complement forms, **subtract** y from x (so you need to negate y and then add). Indicate whether an overflow occurred (and show clearly how you know), and convert the solution back to base 10 (even when an overflow occurred and the solution is wrong, i.e., your answer should fit in the 8-bit representation of that form.).

(a) $x = 10, y = -13$

Answer: $x = 00001010, y = 11110011$, so $-y = 00001101$ and
 $x - y = (00010111)_2 = (23)_{10}$

(b) $x = 117, y = 35$

Answer: $x = 01110101, y = 00100011$, so $-y = 11011101$ and
 $x - y = (01010010)_2 = (82)_{10}$ (**no overflow** since last 2 carries are both 1)

(c) $x = 117, y = -35$

Answer: $x = 01110101, y = 11011101$, so $-y = (00100011)$ and
 $x - y = (10011000)_2 = (-104)_{10}$ (**overflow** since last 2 carries differ)

(d) $x = -117, y = 35$

Answer: $x = 10001011, y = 00100011$, so $-y = 11011101$ and
 $x - y = (01101000)_2 = (104)_{10}$ (**overflow** since last 2 carries differ)

(e) $x = -117, y = 11$

Answer: $x = 10001011, y = 00001011$, so $-y = 11110101$ and
 $x - y = (10000000)_2 = (-128)_{10}$ (**no overflow** since the last two carries are the same)

4. Given the bit pattern 1010 1101 0001 0000 0000 0000 0000 0010 , what does it represent, assuming that it is

(a) a two's complement integer?

Answer: (We first note that by grouping the bits into groups of 4, each 4-bit sequence can be viewed as a base-16 number between 0 and 15. The value represented by this bit sequence is then multiplied by $(2^4)^i$, or 16^i , where i is its position from the right (starting at 0). The one exception is the leftmost ($i = 7$) grouping, which needs special consideration when the uppermost bit needs special treatment (e.g., signed magnitude).)

i) The most significant bit is 1, then flip all bits plus 1 and we get $-X = 0101\ 0010\ 1110\ 1111\ 1111\ 1111\ 1111\ 1110 = 5(16^7) + 2(16^6) + 14(16^5) + 15(16^4) + 15(16^3) + 15(16^2) + 15(16^1) + 14(16^0) = 1391460350$, hence $X = -1391460350$.

(b) an unsigned integer?

Answer: $= 10(16^7) + 13(16^6) + 1(16^5) + 2(16^0) = 2903506946$

(c) a single precision (32-bit) floating-point number?

Answer: $= (-1) \times (1 + 2^{-3} + 2^{-22}) \times (2^{-37}) = -8.185454 \times 10^{-12}$

5. Represent the following numbers, given here in binary form, as floating point numbers using IEEE 754 floating-point standard representation:

(a) 11010.1110

Answer: 0100 0001 1101 0111 0000 0000 0000 0000

(b) -11011.10

Answer: 1100 0001 1101 1100 0000 0000 0000 0000

(c) 0.000101

Answer: 0011 1101 1010 0000 0000 0000 0000 0000

(d) -1.010101

Answer: 1011 1111 1010 1010 0000 0000 0000 0000

(e) 10000000010

Answer: 0100 0100 1000 0000 0100 0000 0000 0000

6. Below is a “weird” encoding for a 3-bit wordsize that encodes only 0 and prime values. Suppose we ask a 3-bit computer to add all possible pairs of values and return the result (in the same encoding, of course). Indiciate which pairs of numbers when summed together produce an overflow.

Bit sequence	value
000	0
001	1
010	2
011	3
100	5
101	7
110	11
111	13

Answer: Recall that overflow simply means that the result is not representable in the current representation. So $1 + 3 = 4$ cannot be represented (because there is no encoding that represents 4, nor can

- $1 + 5, 1 + 7, 1 + 11, 1 + 13,$
- $2 + 2, 2 + 7, 2 + 13,$
- $3 + 3, 3 + 5, 3 + 7, 3 + 11, 3 + 13$
- $5 + 5, 5 + 7, 5 + 11, 5 + 13,$
- $7 + 7, 7 + 11, 7 + 13,$
- $11 + 11, 11 + 13,$
- $13 + 13$

or any of the above with the order of numbers flipped e.g., $5 + 1, 7 + 1$, etc.

7. **OPTIONAL** Challenge Question (with exam-like “twist”): Given a k -bit wordsize, explain which statements are true, and which ones are false, and why:

- (a) It is possible to construct a representation that ranges between integer values n_1 and n_2 where $n_2 = n_1 + 2^k - 1$. For instance, unsigned has $n_1 = 0, n_2 = 2^k - 1$ and 2’s-complement has $n_1 = -2^{k-1}$ and $n_2 = 2^{k-1} - 1$. But can it be done, for example, with $n_1 = 12$, or $n_1 = -3$ What restrictions are there on n_1 as a function of k ?

Answer: There are no restrictions. It can be done for any value of n_1 . A wordsize of k lets you assign 2^k different values to the 2^k possible combinations of k bits.

- (b) For what values of n_1 will the BAA algorithm produce a correct result (modulo 2^k)? Why? (I'd probably give more hints for this question on an exam)

Answer: The BAA algorithm will work correctly for any value of n_1 . Informally, we are just “shifting” our view of the ranges of values covered on the pinwheel, the BAA algorithm will still “spin” around the pinwheel the same way (an answer acceptable on an exam).

More formally: This is because if X and Y are k -bit values, we know that $X + Y = Z$ is correct in unsigned form. In an alternate representation, Letting v_α represent the value of a k -bit sequence α in unsigned form. Then in another representation, X would have the value $V_X + W$ where $W = 2^k C$ for some constant C , and Y would have the value $V_Y + W'$ where $W' = 2^k C'$ for some constant C' (and in fact $C' = C$ or $C' = C + 1$ or $C' = C - 1$). So $X + Y = Z$ being correct in unsigned form (modulo 2^k) means $V_X + V_Y = V_Z \bmod 2^k$. But then $V_X + C2^k + V_Y + C'2^k = Z \bmod 2^k$ will also hold true since we are just adding constants that are multiples of 2^k , which have no effect mod 2^k .

- (c) Hard question: Come up with a general algorithm to detect overflow. Note that both unsigned and 2's-complement have easy methods to detect overflow (i.e., for unsigned, check the highest order carry, for 2's-complement, compare the two highest order carries. So the approach changes depending on the range of values assigned to the bit-sequences.

Answer: Suppose we are adding two values X and Y . The observation is that if X and Y are both positive, then the result Z should be larger than X and Y . If X is positive and Y is negative, then the result should be larger than Y and smaller than X . If X and Y are both negative, then the result should be smaller than both X and Y .

First, consider the case when X is positive and Y is negative (or vice versa). Since the result lies between X and Y , it cannot cause overflow (the value must be representable if lies between them), so for this case, we can simply state “no overflow”

Now, consider when X and Y are both positive. If we have overflow, it means that $X + Y$ is not representable (i.e., bigger than n_2 , but the BAA algorithm will return the correct result mod 2^k). Think about when overflow occurs for the unsigned case: the result of $X + Y$ will be some Z that is correct mod 2^k , and since $Y < 2^k$, the computation will rotate around the pinwheel, but stop short of X at a value smaller than X (because $Y < 2^k$) so it “goes past 0” (else there wouldn't be overflow). But it would require $Y = 2^k$ to loop all the way back around to X , so it stops short of X . So for unsigned, we stop at a value smaller than X . Clearly this is an overflow, as $X + Y$ when both are positive will not equal something smaller than X . So we just have to check whether the result is less than either of the values being summed, and if so, it must be an overflow.

This same logic applies in any representation where both X and Y represent positive values, because it's the same movement on the pinwheel - it's correct modulo 2^k , but if the resulting sum is smaller than X , it's overflow. Otherwise, it's correct.

A similar argument can be made for when X and Y are both negative: if the result is positive, or is negative but has smaller magnitude than X or Y , there is an overflow.

NOTE: When $n_1 \geq 2^k$, overflow will definitely happen, i.e., even the smallest addition $n_1 + n_1 \geq n_1 + 2^k$, and since $n_2 = n_1 + 2^k - 1$, we must have overflow. Similarly, when $n_2 \leq -2^k$, we also must have overflow, since even the largest addition $n_2 + n_2 \leq n_2 - 2^k$, and since $n_1 = n_2 - 2^k + 1$, we must have overflow.